VIDEO CHAT WEBSITE

Link: https://vid-at.herokuapp.com/

By: Chhanam Thorat

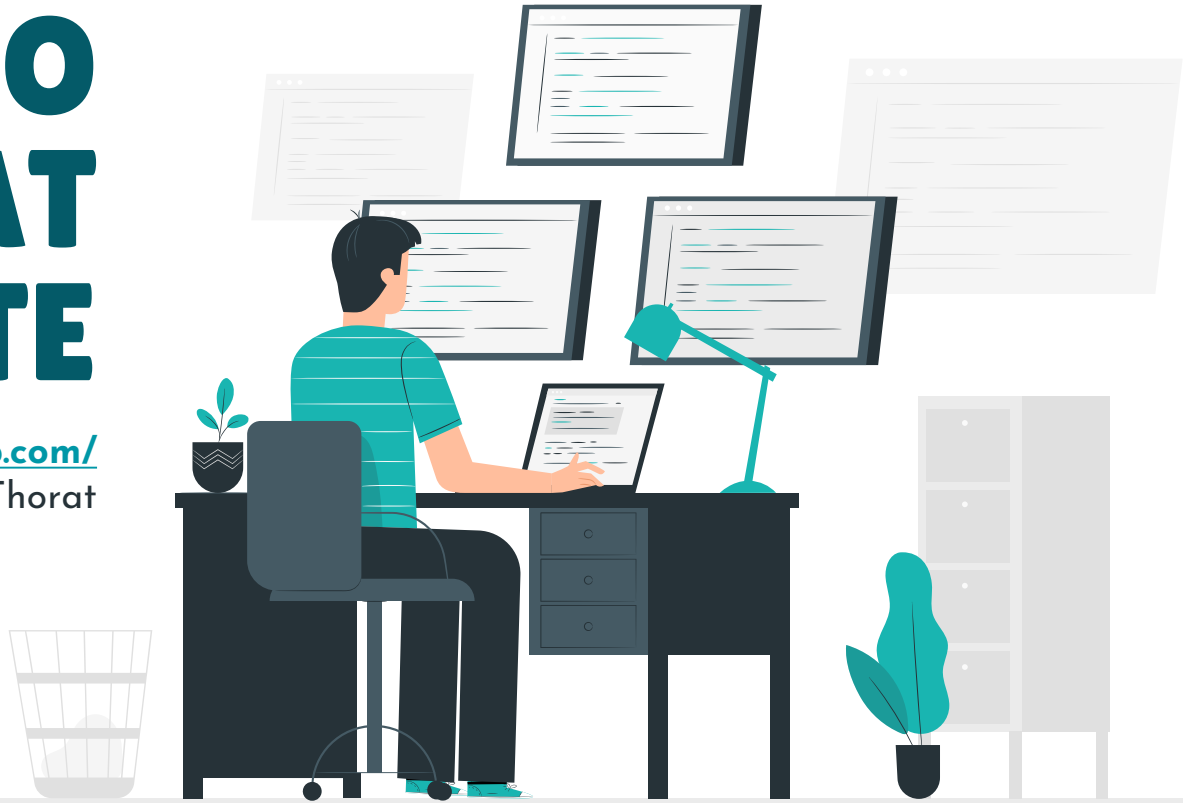


TABLE OF CONTENTS

01 **Introduction**
What is Video-chat website and its need

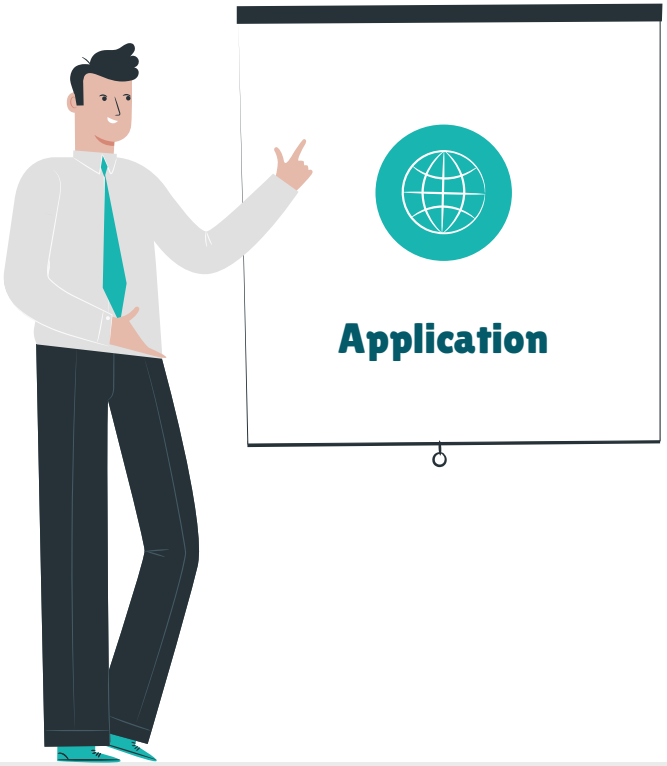
02 **Native Device Apps**
Existing traditional technology

03 **Website using WebRTC**
Our proposed method

04 **Architecture**
The design and layout overview

05 **Implementation**
Development logic of the Video-chat website

06 **Conclusion**
Summary and Future Scope



Introduction

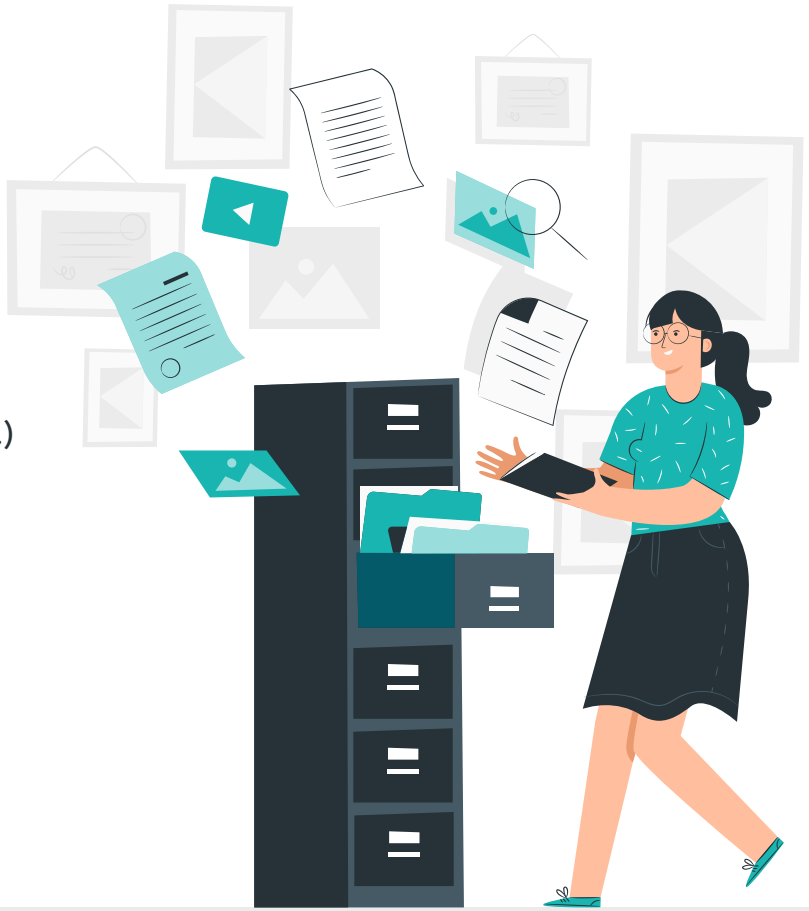
We can establish a session between two peer client browsers and send video stream data over HTTP protocol.

The application being hosted over the internet, we save our internal device memory space.

And with the browsers being available by default on all desktop and mobile devices we can access it anytime and anywhere.

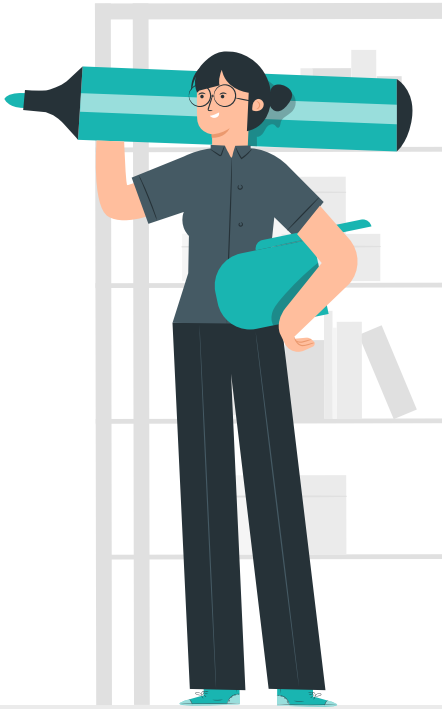
NATIVE DEVICE APPS

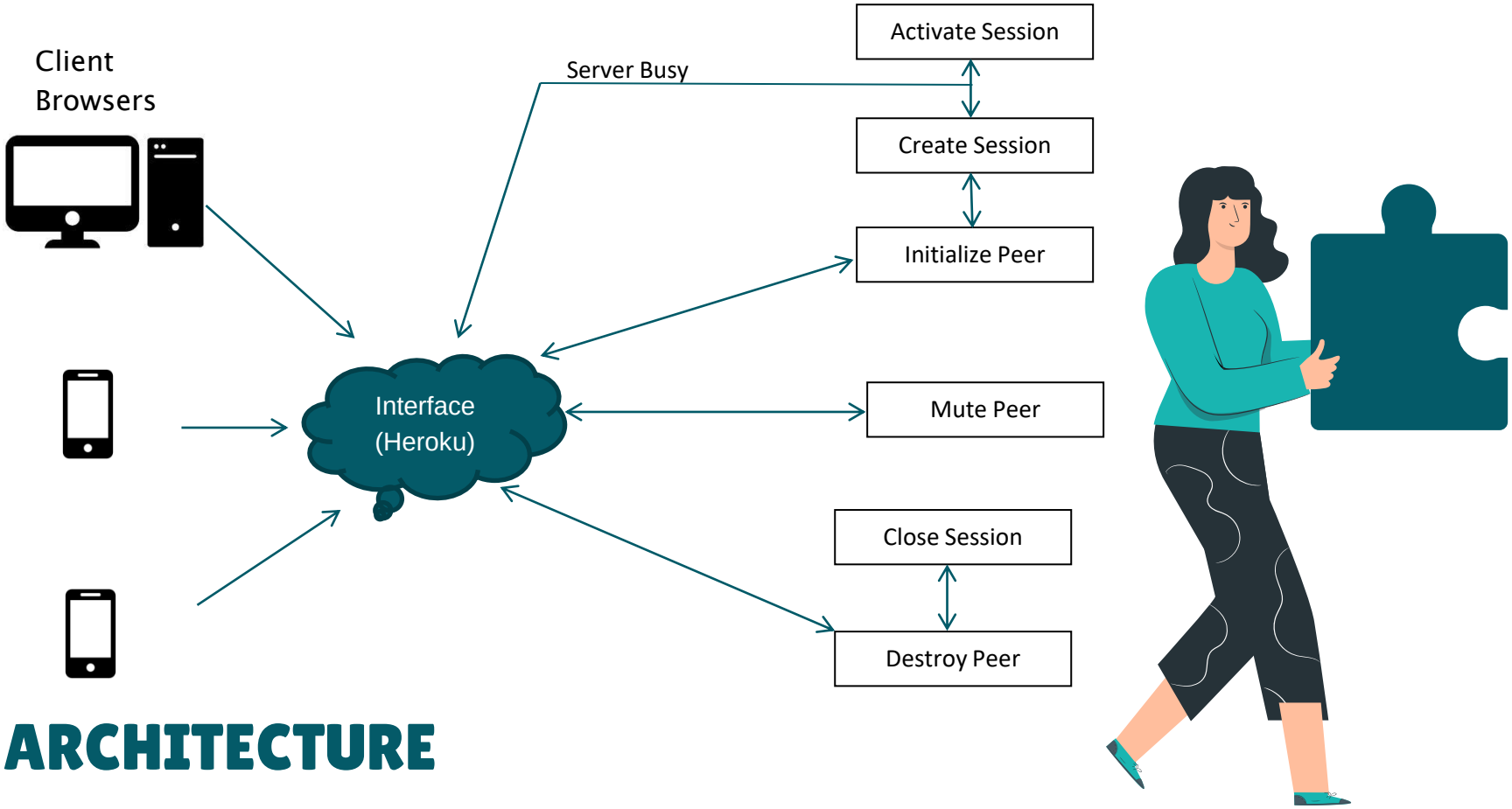
- Client side load and processing is heavy
- Cross platform support (IOS, Android, Linux, etc.)
- Advanced functions – interact with other apps, push notifications, etc.
- Higher computing performance
- More security options
- Reliability and robustness
- Device portability



WEBSITE USING WebRTC

- Client side load and processing is heavy
- Use web-technologies – HTML, CSS, JavaScript
- Cheaper than native app
- Faster to build and update
- No need of building different versions for different platforms
- Cross-Browser compatibility
- Search Engine Optimization for Brand
- Less client-side processing needed
- Decreased memory consumption





ARCHITECTURE

IMPLEMENTATION

Front-end

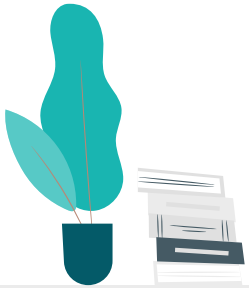


JavaScript - ES6
HTML
CSS



Back- end

WebRTC
Node.js
Express JS
Socket.io





ENTITIES

01

Peer

Another client browser instance that we want to establish a connection with for data streaming.

02

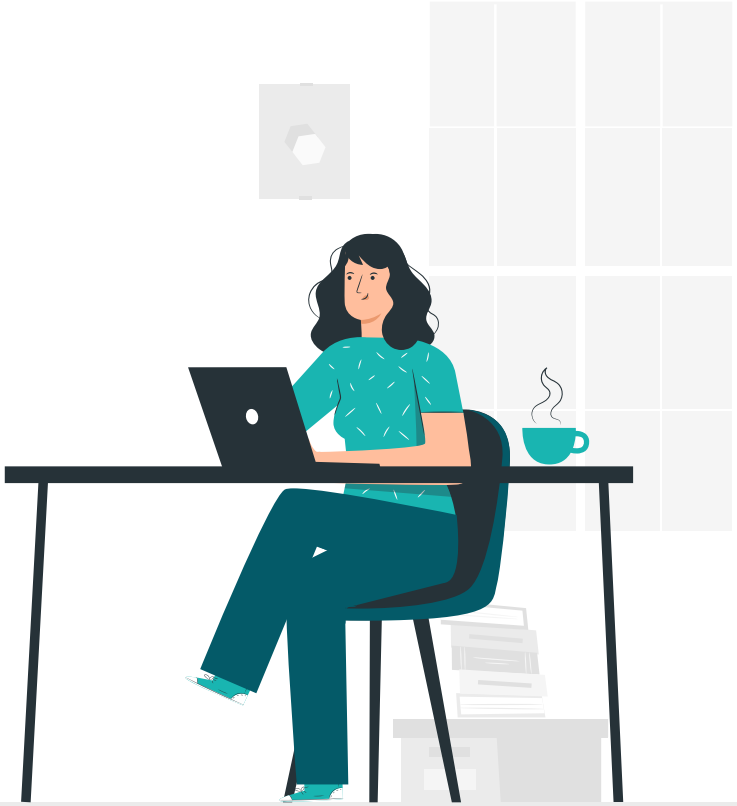
User

Refers to Client itself accessing the website.

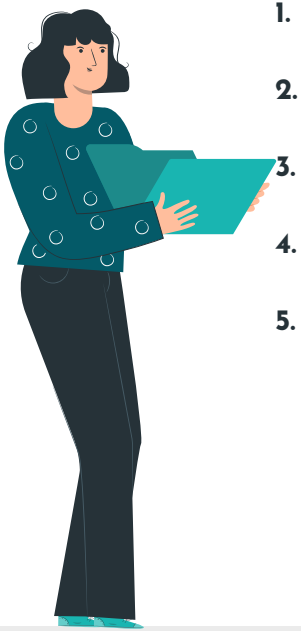
03

Session

Create a common medium for clients to communicate.



METHODS



1. **CreatePeer()** – Initialize a client's request as Peer and create a Video object ready for streaming on the website.
2. **SendOffer()** – The former initialized peer sends offer to another peer to connect to the active session
3. **SendAnswer()** - The later initialized peer sends answer of acknowledgement of offer back to the former initialized peer.
4. **SessionActive()** – Establishes a secure connection over HTTPS and prevents third peer client from disturbing the active session.
5. **Disconnect()** – Destroys the peer object on browser close event and remove the video component from page.





CONCLUSION

With WebRTC, we can stream the data over the internet using HTTP's support for security.

We created a session for two client browsers sending an offer and answer.

We also handled any additional peer request for connection. In future, we can convert the website to function as an application using PWA.